

SEARCHING

PROBLEM 4

QUESTION:

DS	Session	Searching	Question Information	Level 1	Challenge 4
Question Description: Ragu has given a range [L, R] to Smith. Smith wants to require to find the number of integers 'X' in the range such that $\text{GCD}(X, F(X)) > 1$ where $F(x)$ is equal to the sum of digits of 'X' in its hexadecimal (or base 16) representation. Example : $F(27) = 1+8=1+11=12$ (27 in hexadecimal is written as 1B) Constraints: $1 \leq T \leq 50$ $1 \leq L$ $R \leq 10^5$ Input Format: The first line contains a positive integer 'T' denoting the number of questions that you are asked. Each of the next 'T' lines contain two integers 'L' and 'R' denoting the range of questions. Output Format: Print the output in a separate lines exactly 'T' numbers as the output.					

Sample Input: 3 1 3 5 8 7 12 Sample output: 2 4 6 Explanation For the first test-case, numbers which satisfy the criteria specified are : 2,3. Hence, the answer is 2. For the second test-case, numbers which satisfy the criteria specified are : 5,6,7 and 8. Hence, the answer is 4. For the third test-case, numbers which satisfy the criteria specified are : 7,8,9,10,11 and 12. Hence, the answer is 6.

Logical Test Cases

Test Case 1	Test Case 2
INPUT (STDIN)	INPUT (STDIN)
3	10
1 3	15 64
5 8	14 100
7 12	9 79
	19 63
	3 83
	5 61
	18 93

EXPECTED OUTPUT 2 4 6	28 98 13 71 12 53 EXPECTED OUTPUT 28 52 45 26 53 37 45 47 36 26
▼ Mandatory Test Cases Test Case 1 keyword int search(int a, int b)	
▼ Complexity Test Cases Test Case 1 CYCLOMATIC COMPLEXITY 2 Test Case 2 TOKEN COUNT 165 Test Case 3 MDCC 27	

SOURCE CODE:

```
#include <stdio.h>

int find_gcd(int x, int y) {
    while (y != 0) {
        int remainder = x % y;
        x = y;
        y = remainder;
    }
    return x;
}

int sum_hex_digits(int val) {
    int acc = 0;
    while (val > 0) {
        acc += (val & 15);
        val >>= 4;
    }
    return acc;
}

int count_special_range(int start, int end) {
    int valid_total = 0;
    for (int num = start; num <= end; num++) {
        int fx = sum_hex_digits(num);
        if (find_gcd(num, fx) > 1) {
            valid_total++;
        }
    }
    return valid_total;
}

int main() {
    int cases;
    if (scanf("%d", &cases) != 1) return 0;
    while (cases--) {
        int low_bound, high_bound;
        scanf("%d %d", &low_bound, &high_bound);
        printf("%d\n", count_special_range(low_bound, high_bound));
    }
    return 0;
}
```

OUTPUT:

```
3
1 3
5 8
7 12
2
4
6
```

```
2
4
6
20 98
13 71
12 53
29
```